Sprawozdanie z projektu procesora w języku VHDL

Grupa: **L2**

Spawozdanie wykonano przez: **Oleksii Izvarin, Illia Kostruba,**

1. Wstęp

1.1. Przedmiot projektu

Przedmiotem niniejszego projektu jest opracowanie, implementacja oraz weryfikacja cyfrowego modelu 16-bitowego procesora. Zaprojektowany układ jest procesorem wielocyklowym, w którym wykonanie pojedynczej instrukcji zostało rozbite na sekwencję kroków realizowanych в kolejnych cyklach zegara. Urządzenie zostało zaprojektowane w języku opisu sprzętu VHDL (Very High Speed Integrated Circuit Hardware Description Language) z myślą o późniejszej syntezie w układach programowalnych FPGA. Architektura procesora opiera się na zestawie rejestrów ogólnego przeznaczenia oraz dedykowanej jednostce sterującej realizującej cykl rozkazowy maszyny.

1.2. Cel projektu

Głównym celem projektu było stworzenie w pełni funkcjonalnej jednostki centralnej (CPU), zdolnej do wykonywania podstawowych operacji arytmetyczno-logicznych, zarządzania przepływem danych między rejestrami a pamięcią zewnętrzną (RAM) oraz obsługi instrukcji sterujących.

Kluczowe cele szczegółowe obejmowały:

- Zaprojektowanie wydajnej Jednostki Arytmetyczno-Logicznej (ALU) obsługującej 24 operacje na liczbach ze znakiem.
- Implementację Pliku Rejestrów wspierającego dwuportowy odczyt oraz operacje specjalne typu Sid (in-place increments/shifts).

- Stworzenie Interfejsu Pamięci zapewniającego stabilną komunikację z pamięcią operacyjną poprzez rejestry MAR i MBR.

- Opracowanie Jednostki Sterującej (Control Unit) w formie automatu stanów (FSM) sterującego fazami Fetch, Decode, Execute oraz Writeback.

1.3. Założenia techniczne

- Procesor został zaprojektowany jako jednostka wielocyklowa, sterowana przez automat stanów (FSM), co pozwala na podział wykonywania każdej instrukcji na kilka faz (Fetch, Decode, Execute, Writeback). Projekt uwzględnia specyficzne wymagania dla zespołu 4-osobowego oraz rozszerzony zestaw instrukcji.
- Kluczowe parametry techniczne:
- Model pracy: Architektura wielocyklowa zapewniająca stabilną realizację operacji w kolejnych cyklach zegara.
- Szerokość szyny danych: 16 bitów.
- Szerokość szyny adresowej: 32 bity, co pozwala na adresowanie rozległej przestrzeni pamięci RAM.
- Format liczb: Stałoprzecinkowy, kodowanie w systemie uzupełnień do dwóch.
- Plik rejestrów (Register File):
    - 20 rejestrów - rozszerzenie bazy o 8 dodatkowych rejestrów (2 na każdego członka zespołu).
    - R0 – R13: rejestry ogólnego przeznaczenia
    - Dedykowane rejestry specjalne: PC (licznik rozkazów), SP (wskaźnik stosu), IR (rejestr instrukcji), AD (rejestr adresowy) oraz rejestry pomocnicze (TMP, ATMP).
    - Wbudowane układy inkrementacji/dekrementacji dla rejestrów PC oraz SP (mechanizm Sid).
- Jednostka Arytmetyczno-Logiczna (ALU):

    - Obsługa 24 unikalnych operacji arytmetycznych, logicznych oraz przesuwnych. Rozszerzenie bazy o 8 dodatkowych rozkazów (2 na każdego członka zespołu).
    - Zestaw flag stanu C (przeniesienie), Z (zero), N (znak/ujemny), V (overflow).
- Interfejs pamięci: Wykorzystanie dedykowanych rejestrów MAR (adresowy) oraz MBR (buforowy) do separacji procesora od magistrali zewnętrznej.

2. Architektura i Lista Instrukcji

2.1. Ogólna struktura procesora

Serce systemu stanowi wielocyklowa ścieżka danych (multi-cycle datapath), w której każda instrukcja (np. ADD, LOAD, SWAP) jest procesowana w oddzielnych taktach zegara (Fetch, Decode, Execute, Write-back). Architektura opiera się na modelu Load/Store, co oznacza, że operacje arytmetyczno-logiczne są wykonywane wyłącznie na rejestrach, a dostęp do pamięci RAM realizowany jest za pomocą dedykowanych instrukcji przesyłania danych.

Główne komponenty systemu:

- **Jednostka Sterująca (Control Unit):** Zarządza przepływem danych za pomocą 13-stanowego automatu FSM.
- **Plik Rejestrów:** Zawiera 14 rejestrów ogólnego przeznaczenia (R0-R13) oraz 6 rejestrów specjalnych (PC, SP, AD, ATMP, TMP, IR).
- **ALU:** Wykonuje operacje na liczbach 16-bitowych i generuje flagi stanu: Zero (Z), Negative (N), Carry (C), Overflow (V).

2.2. Format Instrukcji

Wszystkie instrukcje procesora mają stałą długość 16 bitów. Podział słowa instrukcji na poszczególne pola zależy od typu operacji, jednak zachowano spójny układ dla większości rozkazów:

Bity	15 - 13	12 - 9	8 - 5	4 - 0

Funkcja	Kod Instrukcji (Instr code)	Rejestr Docelowy (RegA)	Rejestr Źródłowy (RegB)	Wartość natychmiastowa / Funkcja ALU

2.3. Lista Rozkazów (Instruction Set)

Poniższa tabela przedstawia główne grupy instrukcji zaimplementowane w jednostce sterującej:

Typ instrukcji	Instr code (bin)	Opis	Przykład
ALU Operations	000	Operacje matematyczne i logiczne (ADD, SUB, AND, itp.)	ADD R1, R2
LOAD	001	Pobranie danych z pamięci pod adres: RegB + Offset	LOAD R1, R2, 4
STORE	010	Zapis danych do pamięci pod adres: RegB + Offset	STORE R1, R5, 0
ADDI	011	Dodawanie wartości natychmiastowej (5-bit) do rejestru	ADDI R1, R2, 15
SWAP	100	Zamiana zawartości dwóch rejestrów	SWAP R1, R2
REG_SHIFT	101	Szybkie operacje wewnątrz rejestrów (SHL, SHR, ROL, ROR)	SHL R1

3. Jednostka Arytmetyczno-Logiczna (ALU)

3.1. Charakterystyka modułu

Jednostka Arytmetyczno-Logiczna (ALU) stanowi serce obliczeniowe procesora. Moduł ten jest czysto kombinacyjny, co oznacza, że wynik obliczeń pojawia się na wyjściu natychmiast po ustabilizowaniu się sygnałów wejściowych. ALU operuje

na dwóch 16-bitowych słowach danych (wejścia A i B) i wykonuje operację określoną przez 5-bitowy kod operacji $OPCODE$.

3.2. Pełny Zbiór Instrukcji (24 Operacje)

Grupa 1: Operacje Arytmetyczne (6 operacji)

- 00000 **(ADD):** Dodawanie $A + B$. Aktualizuje flagi C (Carry) i V (Overflow).
- 00001 **(SUB):** Odejmowanie $A - B$. Realizowane jako dodawanie liczby przeciwnej ($A + !B + 1$).
- 00010 **(MUL):** Mnożenie signed. Wykorzystuje 32-bitowy wynik pośredni, zwracając młodsze 16 bitów.
- 00011 **(DIV):** Dzielenie signed. Zawiera sprzętową kontrolę dzielenia przez zero (ustawia flagę V).
- 10100 **(INC):** Inkrementacja $A + 1$. Szybkie zwiększanie wartości o jeden.
- 10101 **(DEC):** Dekrementacja $A - 1$. Szybkie zmniejszanie wartości o jeden.

Grupa 2: Operacje Relacyjne i Porównania (6 operacji)

- 00100 **(SGT):** Set Greater Than. Wynik = 1, jeśli $A > B$, w przeciwnym razie 0.
- 00101 **(SLT):** Set Less Than. Wynik = 1, jeśli $A < B$, w przeciwnym razie 0.
- 00110 **(MAX):** Wybór większej wartości z pary (A, B).
- 10010 **(MIN):** Wybór mniejszej wartości z pary (A, B).
- 10110 **(SEQ):** Set Equal. Wynik = 1, jeśli $A = B$.
- 10111 **(SNE):** Set Not Equal. Wynik = 1, jeśli $A != B$.

Grupa 3: Operacje Logiczne (4 operacje)

- 00111 **(NOT):** Negacja bitowa wejścia A (Inwersja).
- 01000 **(OR):** Alternatywa bitowa.
- 01001 **(AND):** Koniunkcja bitowa.
- 01010 **(XOR):** Alternatywa wykluczająca.

Grupa 4: Przesunięcia i Rotacje (8 operacji)

Ta grupa pozwala na manipulację pozycjami bitów, co jest kluczowe dla optymalizacji matematycznej i kodowania.

- 01011 **(SRA_1):** Arytmetyczne przesunięcie w prawo o 1 bit (zachowuje bit znaku).
- 01100 **(SLL_1):** Logiczne przesunięcie w lewo o 1 bit (wstawia 0 na końcu).
- 01101 **(SRAV):** Arytmetyczne przesunięcie w prawo o B bitów (Variable Shift).
- 01110 **(SLLV):** Logiczne przesunięcie w lewo o B bitów (Variable Shift).
- 01111 **(ROR):** Rotacja w prawo (bit z pozycji 0 wraca na pozycję 15).
- 10000 **(ROL):** Rotacja w lewo.
- 10001 **(REV):** Reverse. Całkowite lustrzane odbicie wektora bitów.
- 10011 **(SRL_1):** Logiczne przesunięcie w prawo o 1 bit (wstawia 0 na pozycji MSB, ignorując znak).

3.3. Zaawansowana Logika Flag Stanu

Wszystkie 24 operacje wpływają na wektor flag w sposób ujednolicony, co zostało zaimplementowane na końcu procesu ALU:

1. **Z (Zero Flag):** $flags_v(0) := '1'$ jeśli result_v to same zera.
2. **N (Negative Flag):** $flags_v(1) := result_v(15)$. Informuje, czy wynik jest ujemny w systemie U2.
3. **C (Carry Flag):** Obliczana selektywnie dla ADD/SUB. Informuje o przeniesieniu poza zakres 16-bitowy.
4. **V (Overflow Flag):** Wykrywa błędy logiczne w arytmetyce liczb ze znakiem (np. dodanie dwóch dużych liczb dodatnich dające wynik "ujemny").

4. Plik Rejestrów (Register File)

4.1. Architektura i zasoby rejestrowe

Plik rejestrów stanowi lokalną, szybką pamięć procesora, umożliwiającą jednoczesny odczyt dwóch operandów oraz zapis jednego wyniku w jednym cyklu zegarowym. Moduł zarządza 16-bitowymi danymi oraz 32-bitowymi adresami.

Struktura wewnętrzna obejmuje:

- **Rejestry ogólnego przeznaczenia (R0 – R13):** 14 rejestrów 16-bitowych służących do przechowywania danych operacyjnych.
- **Rejestr Instrukcji (IR):** Przechowuje aktualnie dekodowany kod operacji.
- **Rejestr Tymczasowy (TMP):** Wykorzystywany w operacjach złożonych, takich jak SWAP.
- **Rejestry adresowe (32-bitowe):**
    - **PC (Program Counter):** Wskazuje adres następnej instrukcji do pobrania.
    - **SP (Stack Pointer):** Wskazuje wierzchołek stosu w pamięci RAM.
    - **AD (Address Register):** Przechowuje adresy danych dla operacji LOAD/STORE.
    - **ATMP:** Rejestr pomocniczy dla operacji na adresach.

4.2. Mechanizm operacji specjalnych Sid

Unikalną cechą zaimplementowanego modułu jest obsługa sygnału **Sid** (Special In-place Operations). Pozwala on na modyfikację wartości wewnątrz pliku rejestrów, bez konieczności przesyłania danych do głównej jednostki ALU.

Kluczowa logika sterowania: Istotnym aspektem projektowym jest warunek aktywacji operacji przesuwnych i rotacyjnych. Mechanizm Sid (dla kodów 0110 - 1001) wykonuje operację modyfikacji wartości rejestru w momencie, gdy sygnał zezwolenia na zapis zewnętrzny **RegWR_en (reg_write_enable) jest ustawiony na '0'**.

Taka konfiguracja zapewnia, że:

- Wewnętrzna logika przesuwna rejestru ma wyłączność na modyfikację danych.
- Nie dochodzi do kolizji z danymi, które mogłyby być jednocześnie wystawione przez ALU lub interfejs pamięci na magistralę wejściową DI.
- Procesor efektywnie wykorzystuje fazę **S_EXECUTE_REG_SHIFT** w architekturze wielocyklowej, co skraca czas wykonania instrukcji sterujących.

Kod Sid	Operacja	Opis
0001	INC PC	Automatyczna inkrementacja licznika programu (PC=PC+1).
0010	INC SP	Zwiększenie wskaźnika stosu (zdjęcie ze stosu).
0011	DEC SP	Zmniejszenie wskaźnika stosu (odłożenie na stos).
0100	INC AD	Inkrementacja AD (AD = AD + 1)
0101	DEC AD	Dekrementacja AD (AD = AD - 1)
0110	SHL	Przesunięcie logiczne wybranego rejestru w lewo.
0111	SHR	Przesunięcie logiczne wybranego rejestru w prawo.
1000	ROL	Rotacja bitowa wybranego rejestru w lewo.
1001	ROR	Rotacja bitowa wybranego rejestru w prawo.

4.3. System komutowania danych (Multiplexing)

Moduł posiada zaawansowaną logikę wyjściową, która w sposób kombinacyjny (bezzwłoczny) wystawia dane na magistrale wewnętrzne procesora:

4.3.1. Dwuportowy odczyt (Szyny BB i BC): Selektory Sbb i Sbc pozwalają na jednoczesne wystawienie dowolnych dwóch rejestrów na wejścia ALU. Wartość 15 na selektorze umożliwia przekierowanie danych bezpośrednio z wejścia DI (tzw. data bypassing).

4.3.2. Selektor adresu (Sa) Sygnał ten steruje wewnętrznym multiplekserem w bloku rejestrów, wybierając jeden z 32-bitowych rejestrów (PC, SP, AD lub ATMP), który zostanie wystawiony na wewnętrzną magistralę adresową. W fazie FETCH, przy ustawieniu Sa na wartość wskazującą na PC, umożliwia to przesłanie adresu instrukcji do rejestru MAR w interfejsie pamięci.

4.4. Zapis danych (RegWR_en)

Zapis do rejestrów jest synchroniczny i odbywa się przy narastającym zboczu sygnału zegarowego clk. Warunkiem zapisu jest stan wysoki sygnału RegWR_en. Dzięki wykorzystaniu funkcji $resize$, dane 16-bitowe z wejścia BA mogą być poprawnie zapisywane do 32-bitowych rejestrów adresowych (PC, SP, AD) poprzez rozszerzenie znaku.

5. Interfejs Pamięci (Memory Interface)

5.1. Rola Modułu w Architekturze

Moduł $memory_interface$ stanowi warstwę abstrakcji sprzętowej (HAL). Jego zadaniem jest odizolowanie wrażliwych sygnałów wewnętrznych procesora od zakłóceń i specyficznych wymagań czasowych zewnętrznej pamięci RAM. Dzięki zastosowaniu rejestrów buforowych, procesor może przygotować adres i dane w jednym cyklu, a interfejs utrzyma je stabilnie przez cały czas trwania cyklu dostępu do pamięci.

5.2. Architektura Rejestrów MAR i MBR

Wewnątrz modułu zaimplementowano dwa kluczowe rejestry, które są synchronizowane sygnałem zegarowym CLK:

- **MAR (Memory Address Register):** 32-bitowy rejestr przechowujący wskaźnik do pamięci. Pozwala on na zaadresowanie do 2^{32} komórek pamięci.
  - Kod VHDL: $if\ CTL_Smar\ =\ '1'\ then\ MAR\ <=\ DP_ADR_IN;\ end\ if;$
- **MBR (Memory Buffer Register):** 16-bitowy rejestr pełniący rolę "śluzy" dla danych.
  - Kod VHDL: $if\ CTL_Smbr\ =\ '1'\ then\ MBR\ <=\ DP_DATA_OUT;\ end\ if;$

5.3. Zarządzanie Magistralą Dwukierunkową

Jednym z najważniejszych aspektów tego modułu jest obsługa portu MEM_DATA typu $inout$. W systemach cyfrowych magistrala danych jest współdzielona. Aby

uniknąć zwarcia (gdy dwa urządzenia próbują wystawić różne napięcia na ten sam drut), zastosowano logikę trzeciego stanu (High Impedance):

$$MEM_DATA <= MBR\ when\ CTL_WR = '1'\ else\ (others => 'Z');$$

- Tryb Zapisu ($CTL_WR = '1'$): Interfejs "pcha" dane z MBR na szynę.

- Tryb Odczytu/Bezczynności: Interfejs przechodzi w stan wysokiej impedancji ($'Z'$), "odłączając się" od szyny i pozwalając pamięci RAM na bezpieczne wystawienie danych.

6. Jednostka Sterująca (Control Unit)

6.1. Rola Jednostki Sterującej

Jednostka sterująca została zaimplementowana jako automat stanów (FSM), który zarządza procesorem w trybie wielocyklowym. Jest odpowiedzialna za dekodowanie instrukcji pobranych z pamięci oraz generowanie sekwencji sygnałów sterujących dla pozostałych modułów (ALU, Pliku Rejestrów, Interfejsu Pamięci). Logika CU została zaimplementowana jako **automat stanów (FSM)**, co zapewnia precyzyjną synchronizację operacji w czasie.

6.2. Automat Stanów (Automat Mealy`ego)

Sercem jednostki sterującej jest proces $state_transition$, który zarządza przejściami między 13 zdefiniowanymi stanami. Każdy stan odpowiada jednemu cyklowi zegarowemu (w przypadku braku opóźnień z pamięci).

Główne etapy cyklu rozkazowego:

1. Faza S_FETCH (Pobranie instrukcji)

W procesorze wielocyklowym etap ten jest podzielony na dwa cykle zegarowe, aby zapewnić stabilną komunikację z pamięcią RAM.

- **S_FETCH_1 (Adresowanie):** Jednostka sterująca wystawia wartość z licznika programów (**PC**) na szynę adresową poprzez ustawienie sygnału

Addr_src_sel na '0'. Następnie aktywowany jest sygnał CTL_Smar, który zatrzaskuje ten adres w rejestrze **MAR** (Memory Address Register) interfejsu pamięci.

- **S_FETCH_2 (Pobranie i Inkrementacja):** Aktywowany zostaje sygnał odczytu CTL_RD. Kod instrukcji z pamięci trafia do rejestru instrukcji (**IR**). Równocześnie generowany jest sygnał **Sid = "0001"**, co wymusza na pliku rejestrów automatyczne zwiększenie wartości PC o jeden ($PC = PC + 1$).

2. Faza S_DECODE (Dekodowanie)

W tym cyklu procesor przestaje komunikować się z pamięcią zewnętrzną i analizuje pobraną zawartość rejestru IR.

- **Analiza Opkodu:** Jednostka sterująca dekoduje 3 najstarsze bity IR(15 downto 13), aby określić ścieżkę wykonawczą (np. 000 dla ALU, 001 dla LOAD).
- **Przygotowanie Operandów:** Selektory Sbb i Sbc są ustawiane na numery rejestrów źródłowych (zakodowane w bitach 12-9 i 8-5), co powoduje wystawienie ich zawartości na wewnętrzne szyny **BB** i **BC**. Dzięki 20 rejestrom ogólnego przeznaczenia (R0-R19), procesor ma dostęp do szerokiego zestawu danych operacyjnych.

3. Faza S_EXECUTE (Wykonanie)

Zależnie od typu instrukcji, w tej fazie następuje faktyczna obróbka danych.

- **Operacje ALU:** Jednostka ALU wykonuje jedną z 24 operacji na podstawie 5-bitowego kodu OPCODE i aktualizuje flagi stanu: **C, Z, N (Znak), V (Overflow)**.
- **Obliczanie Adresu Pamięci:** Dla instrukcji LOAD/STORE, ALU sumuje wartość rejestru bazowego z 5-bitową wartością natychmiastową (Offset), a wynik jest przesyłany do rejestru MAR.
- **Operacje Sid (Przesunięcia/Rotacje):** Wykonywane są bezpośrednio w pliku rejestrów. **Kluczowa techniczna uwaga:** operacje te (SHL, SHR, ROL, ROR) są realizowane tylko wtedy, gdy sygnał **reg_write_enable wynosi '0'**, co zapobiega konfliktom z zewnętrznym zapisem danych.

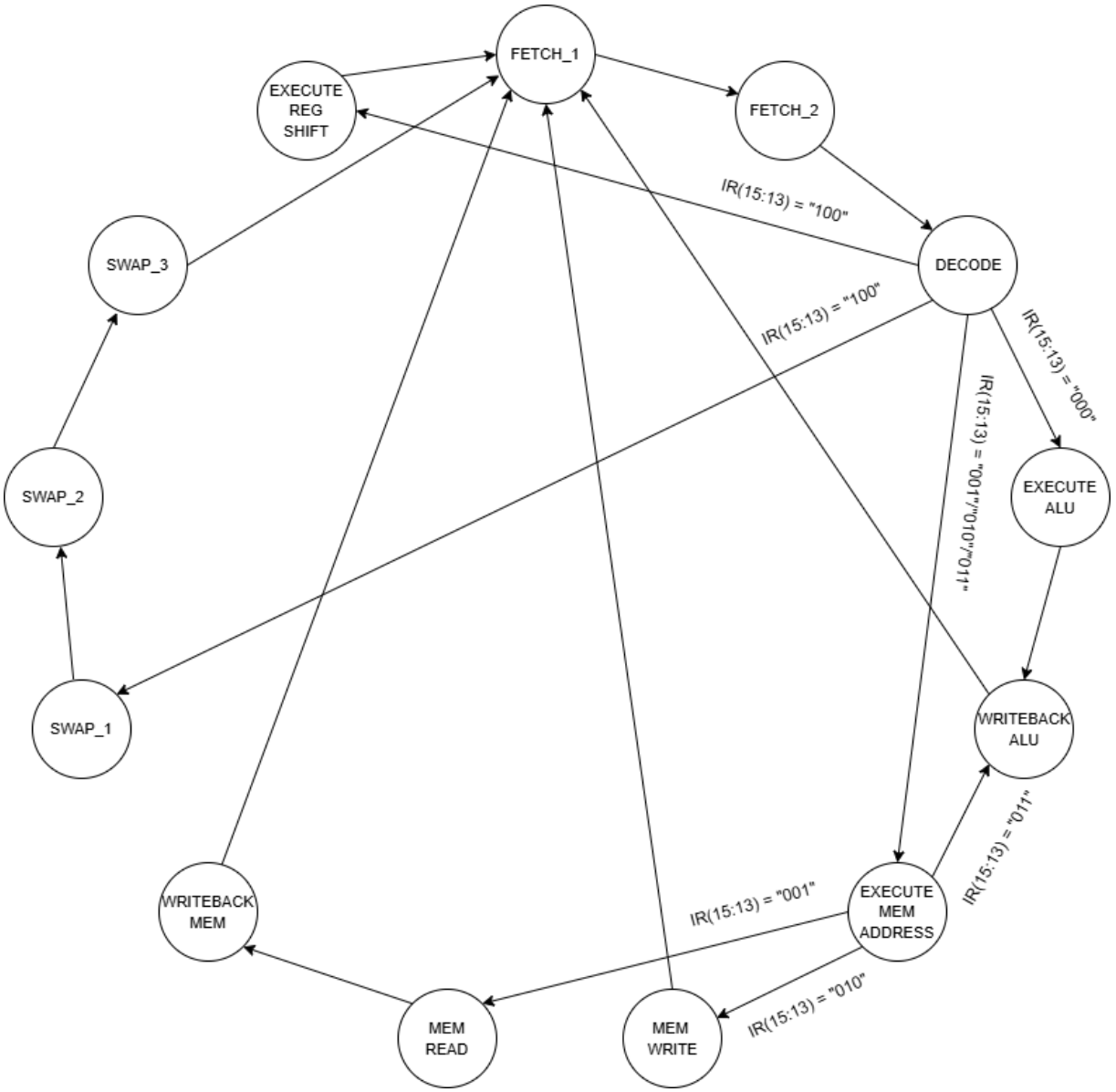
4. Faza S_WRITEBACK (Zapis zwrotny)

Ostatni etap cyklu instrukcji, w którym wynik zostaje trwale zapamiętany.

- **Zapis do Rejestru:** Jednostka sterująca aktywuje sygnał RegWR_en (reg_write_enable = '1').
- **Wybór Źródła:** Sygnał RegWR_src_sel decyduje, czy do rejestru docelowego trafi wynik z ALU (dla instrukcji arytmetycznych), czy dane z pamięci RAM (dla instrukcji LOAD).

Logika przejść między poszczególnymi fazami cyklu rozkazowego została przedstawiona na poniższym grafie. Ilustruje on, jak na podstawie bitów instrukcji IR(15 downto 13) jednostka sterująca wybiera odpowiednią ścieżkę wykonawczą.

GROUPS:	Reister File Control					ALU control		Memory_inteface control						Write-back control
STATES/SIGNALS:	Sba	Sbb	Sbc	Sid	RegWR_en	ALU_opcode	ALU_src_B_sel	Sa	Addr_src_sel	CTL_Smar	CTL_Smbr	CTL_RD	CTL_WR	RegWR_src_sel
S_FETCH_1	31	-	-	-	0	-	-	01	0	1	-	-	0	-
S_FETCH_2	01111	-	-	0001	1	-	-	-	-	-	-	1	0	1
S_DECODE	31	IR(12:9)	IR(8:5)	-	0	-	-	-	-	-	-	-	0	-
S_EXECUTE_ALU	31	IR(12:9)	-	-	0	IR(4:0)	0	-	-	-	-	-	0	-
S_WRITEBACK_ALU	0 & IR(12:9)	-	-	-	1	-	-	-	-	-	-	-	0	0
F IR(15:13)="011" (ADD	31	-	-	-	0	00000	1	-	-	-	-	-	0	-
ELSE	31	IR(12:9)	IR(8:5)	-	0	IR(4:0)	0	-	-	-	-	-	0	-
S_EXECUTE_MEM_ADDF	31	IR(8:5)	-	-	0	00000	1	-	1	1	-	-	0	-
F IR(15:13)="010" STOR	31	-	IR(12:9)	-	0	-	-	-	-	-	1	-	0	-
S_MEM_READ	31	IR(8:5)	-	-	0	00000	1	-	1	-	-	1	0	-
S_MEM_WRITE	31	IR(8:5)	IR(12:9)	-	0	00000	1	-	1	-	-	-	1	-
S_WRITEBACK_MEM	0 & IR(12:9)	IR(8:5)	-	-	1	00000	1	-	1	-	-	1	0	1
S_SWAP_1	1110	IR(12:9)	IR(12:9)	-	1	01000	-	-	-	-	-	-	0	-
S_SWAP_2	0 & IR(12:9)	IR(8:5)	IR(8:5)	-	1	01000	-	-	-	-	-	-	0	-
S_SWAP_3	0 & IR(8:5)	1110	1110	-	1	01000	-	-	-	-	-	-	0	-
S_EXECUTE_REG_SHIFT	0 & IR(12:9)	-	-	IR(3:0)	0	-	-	-	-	-	-	-	0	-



6.3. Szczegółowy opis przejść dla grup instrukcji

W zależności od dekodowanego Opkodu, automat wybiera odpowiednią ścieżkę wykonawczą:

- **Ścieżka ALU (Opcode 000):**

S_DECODE -> S_EXECUTE_ALU -> S_WRITEBACK_ALU -> S_FETCH_1.

- **Ścieżka LOAD (Opcode 001):**

S_DECODE -> S_EXECUTE_MEM_ADDR (obliczanie adresu) -> S_MEM_READ -> S_WRITEBACK_MEM -> S_FETCH_1.

- **Ścieżka STORE (Opcode 010):**

S_DECODE -> S_EXECUTE_MEM_ADDR -> S_MEM_WRITE -> S_FETCH_1.

- **Ścieżka SWAP (Opcode 100):**

S_DECODE -> S_SWAP_1 -> S_SWAP_2 -> S_SWAP_3 -> S_FETCH_1.

Realizowana w trzech krokach (*S_SWAP_1, 2, 3*), aby umożliwić bezpieczną zamianę wartości między rejestrami przy użyciu rejestru tymczasowego TMP.

- **Ścieżka EXECUTE_REG_SHIFT (Opcode 101):**

S_DECODE -> S_EXECUTE_REG_SHIFT -> S_FETCH_1.

6.4. Logika wyjściowa (Control Signals)

W każdym stanie jednostka sterująca ustawia wartości sygnałów, które decydują o zachowaniu procesora. Kluczowe sygnały to:

- **RegWR_en:** Zezwolenie na zapis do rejestrów (aktywne tylko w stanach Writeback).
- **ALU_src_B_sel:** Decyduje, czy drugim argumentem dla ALU jest rejestr (0), czy wartość natychmiastowa z instrukcji (1).
- **Addr_src_sel:** Wybiera źródło adresu dla pamięci (PC dla instrukcji, ALU_result dla danych).
- **CTL_RD / CTL_WR:** Sygnały bezpośrednio sterujące szyną sterującą pamięci RAM.

7. Weryfikacja i Symulacje

7.1. Środowisko testowe (Testbench)

Weryfikacja procesora została przeprowadzona przy użyciu dedykowanego środowiska testowego (Testbench) zapisanego w pliku $processor_tb.vhd$. Środowisko to symuluje otoczenie sprzętowe procesora, w tym:

- **Generator sygnału zegarowego (CLK):** Wytwarzający stabilny impuls taktujący.
- **Model pamięci RAM:** Inicjalizowany gotowym programem w formacie binarnym, co pozwala na obserwację wykonywania realnych instrukcji.
- **Sygnał Reset:** Inicjujący procesor i ustawiający automat stanów w fazę *S_FETCH_1.*

7.2. Scenariusz testowy

W celu pełnej weryfikacji poprawności działania procesora wielocyklowego, opracowano szczegółowy scenariusz testowy zapisany w pliku tb_processor_top.vhd. Scenariusz składa się z sekwencji 13 instrukcji, które sprawdzają każdy kluczowy moduł systemu.

Szczegółowy przebieg testu:

1. Inicjalizacja (4 instrukcje ADDI): * Ustawienie bazy $R0 = 0$.

   Załadowanie wartości do rejestrów: $R1 = 10, R2 = 20, R3 = 5$. Pozwala to na sprawdzenie mechanizmu sign extension dla 5-bitowych wartości natychmiastowych.

2. Operacje ALU (4 instrukcje): * ADD: $R1 = 10 + 20 = 30$.

   XOR: $R2 = 20 \oplus 5 = 17$.

   REV: Odwrócenie bitów w $R3$ (sprawdzenie bloku operacji bitowych).

   MIN: $R1 = min(30, 17) = 17$. Na tym etapie weryfikowane jest poprawne ustawianie flag stanu (C, Z, N, V).

3. Ruch danych (2 instrukcje): * STORE: Zapis wartości 17 z rejestru $R1$ do pamięci RAM pod adres 27 ($R0 + 27$).

   LOAD: Odczyt tej samej wartości z adresu 27 do rejestru $R4$. Potwierdza to poprawną współpracę rejestrów MAR i MBR.

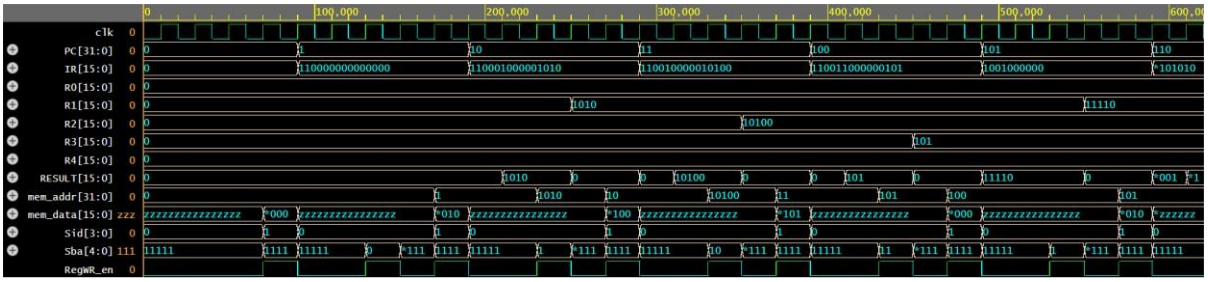
4. Zamiana danych (1 instrukcja SWAP): * Zamiana zawartości $R2$ i $R3$ w celu weryfikacji 3-taktowej ścieżki automatu FSM z wykorzystaniem rejestru tymczasowego TMP.

5. Operacje rejestrowe Sid (2 instrukcje): * SHL: Przesunięcie $R4$ w lewo ($17 \ll 1 = 34$).

   ROR: Rotacja $R4$ w prawo ($34\, rot\, 1 = 17$).

   Operacje te wykonują się bezpośrednio w pliku rejestrów przy sygnale $RegWR_en = '0'$.

7.3. Analiza przebiegów czasowych (Waveforms)



Note: To revert to EPWave opening in a new browser window, set that option on your profile page.

W celu zapewnienia pełnej przejrzystości wyników, weryfikację procesora przedstawiono na dwóch typach przebiegów: binarnym – pozwalającym na analizę kodów instrukcji i sygnałów sterujących, oraz dziesiętnym – ułatwiającym śledzenie zmian wartości w rejestrach roboczych.

1. Procedura startowa i Reset (0 – 60 ns):

- Przez pierwsze 60 000 ps aktywny sygnał reset wymusza stan zerowy na liczniku programu PC oraz rejestrze instrukcji IR. W widoku dziesiętnym potwierdzono, że wszystkie rejestry ogólnego przeznaczenia (R0–R13) są poprawnie wyczyszczone przed rozpoczęciem programu.

2. Sekwencja ładowania danych (ADDI):

Każda instrukcja zajmuje około **100 ns** (5 taktów zegara), co jest charakterystyczne dla zaimplementowanej architektury wielocyklowej.

- **PC=1 (ADDI R1, 10):** W widoku binarnym widać kod 1010 na szynie RESULT, natomiast widok dziesiętny potwierdza, że w rejestrze **R1** pojawia się wartość **10**.
- **PC=2 (ADDI R2, 20):** Rejestr **R2** przyjmuje wartość **20**.
- **PC=3 (ADDI R3, 5):** Rejestr **R3** przyjmuje wartość **5**.

3. Weryfikacja operacji matematycznych (ADD, XOR):

- **ADD R1, R2 (PC=4):** Procesor sumuje wartości z R1 (10) i R2 (20). Widok dziesiętny wyraźnie pokazuje zmianę w rejestrze **R1 z 10 na 30**.
- **XOR R2, R3 (PC=5):** Wykonanie operacji $20 \oplus 5$. Wynik **17** pojawia się na szynie RESULT i zostaje zapisany w rejestrze **R2**.

4. Specyfika sygnałów sterujących i bezpieczeństwo danych:

- **Sygnał Sba [4:0]:** Na obu wykresach zaobserwowano, że przez większość cyklu sygnał ten utrzymuje wartość **31 (11111)**. Jest to zdefiniowany stan jałowy (Idle), który blokuje zapis do rejestrów roboczych w fazach pobierania i dekodowania. Zmiana na właściwy numer rejestru (np. 1 lub 2) następuje wyłącznie w fazie **Writeback**, co gwarantuje spójność danych.
- **Precyzyjna synchronizacja:** Wartość Sba zmienia się na właściwy numer rejestru docelowego (np. 00001 dla R1) **tylko na jeden takt zegara**. Dzieje się to w fazie **Writeback**, ściśle synchronicznie z aktywacją sygnału **RegWR_en**. Takie rozwiązanie gwarantuje, że dane zostaną zatrzaśnięte w pliku rejestrów dopiero po pełnej stabilizacji wyniku na wyjściu ALU, eliminując ryzyko zapisu przypadkowych wartości.
- **Sygnał Sid [3:0]:** Widoczne regularne impulsy o wartości **0001 (INC PC)** w każdym cyklu pobierania instrukcji, co potwierdza poprawne działanie sprzętowego licznika rozkazów wewnątrz pliku rejestrów.

- **Magistrala mem_data:** Stabilność szyny potwierdza stan wysokiej impedancji (zzz), w który przechodzi ona natychmiast po pobraniu kodu operacji, eliminując ryzyko konfliktów na magistrali danych.

Druga faza testu, udokumentowana wyżej, obejmuje zakres czasu od **650 ns do 1200 ns**. W tej części zweryfikowano poprawność operacji modyfikacji bitów oraz kluczowy mechanizm zapisu i odczytu z pamięci RAM.

1. Operacje ALU i manipulacja bitami (PC 6 – 7):

- **PC = 6 (REV R3):** Procesor wykonuje instrukcję lustrzanego odbicia bitów dla rejestru R3. W widoku dziesiętnym widać, że wartość w **R3** zmienia się z 5 na **40960** (co odpowiada binarnemu odwróceniu ...00101 na 10100...), co potwierdza sprawność bloku operacji bitowych.
- **PC = 7 (MIN R1, R2):** Wyznaczenie mniejszej wartości z pary R1 (30) i R2 (17). Na szynie RESULT pojawia się wartość **17**, która zostaje ponownie zapisana w rejestrze **R1**.

2. Komunikacja z pamięcią RAM (PC 8 – 9): Jest to moment weryfikacji architektury **Load/Store** oraz rejestrów MAR i MBR.

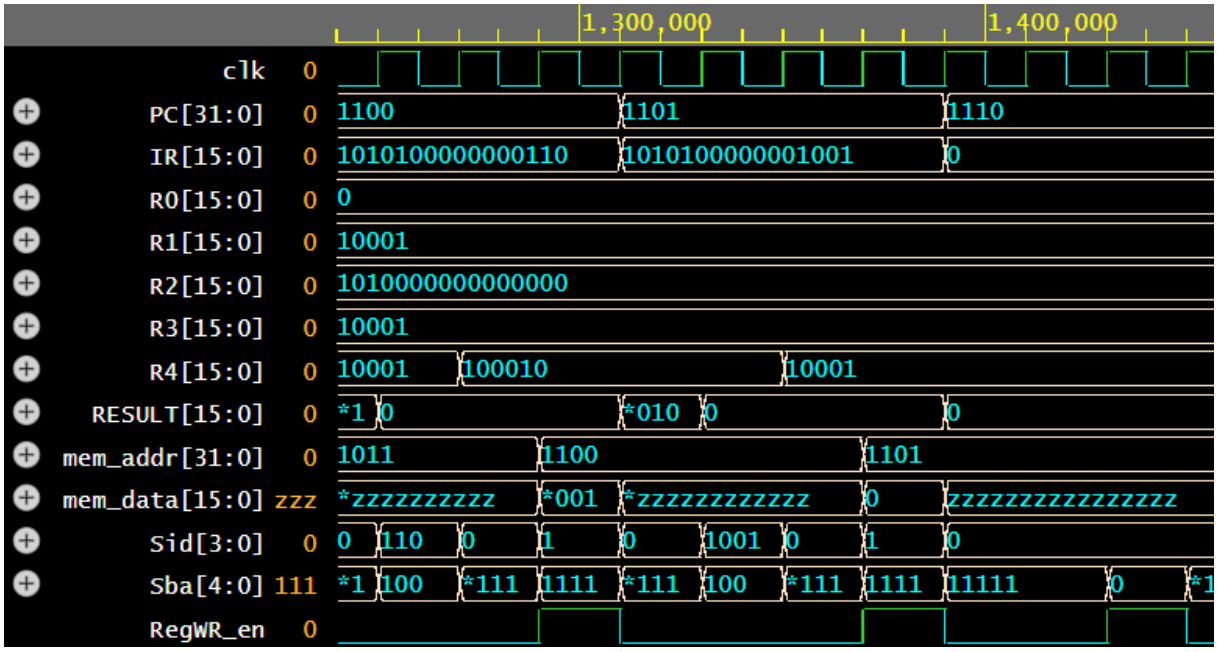
- **PC = 8 (STORE [R0+27], R1):** Zapis wyniku do pamięci RAM pod adres 27.
    - Sygnał mem_addr przyjmuje wartość **27** (wynik sumowania R0 + Immediate 27).

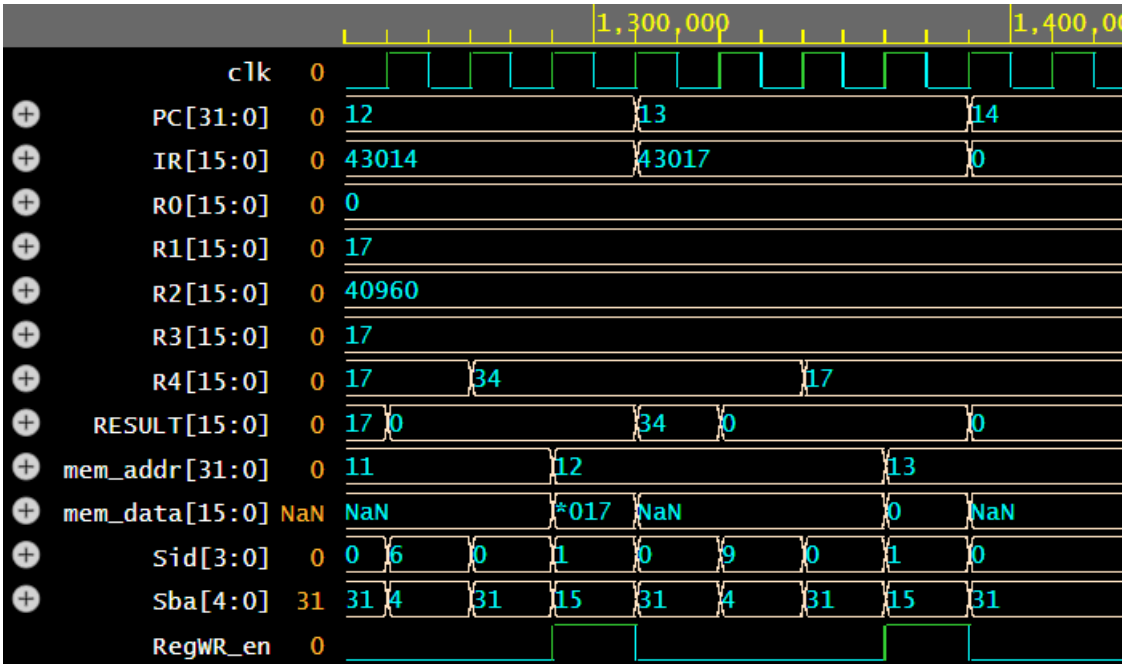
    - Szyna mem_data wystawia wartość **17**. Widok binarny potwierdza, że interfejs pamięci utrzymuje te dane stabilnie przez cały cykl zapisu.

- **PC = 9 (LOAD R4, [R0+27]):** Odczyt danych z RAM.
    - Procesor ponownie adresuje komórkę **27**.
    - Wartość 17 zostaje pobrana z szyny danych i przekazana do rejestru **R4**, co widać jako zmianę wartości R4 z 0 na **17** w widoku dziesiętnym.
- **PC = 10 (SWAP R2, R3)**: Zamiana zawartości między rejestrami R2 i R3.
    - Operacja realizowana jest w trzech krokach (fazy S_SWAP_1, 2, 3) z wykorzystaniem wewnętrznego rejestru tymczasowego TMP.
    - Widok dziesiętny potwierdza poprawną zamianę: rejestr R2 przyjmuje wartość z R3, a R3 otrzymuje poprzednią wartość z R2.

3. Synchronizacja sygnałów i bezpieczeństwo magistrali:

- **Sygnał Sba i RegWR_en:** Na wykresach widać, że adres zapisu Sba wskazuje rejestr docelowy (np. 1 lub 4) **tylko na jeden takt zegara** w fazie Writeback, synchronicznie z impulsem RegWR_en. Przez resztę czasu sygnał utrzymuje bezpieczną wartość **31**.
- **Trzeci stan (High-Z):** Szyna mem_data przechodzi w stan wysokiej impedancji (zzz) natychmiast po operacjach pamięciowych, co widać na wykresie binarnym. Zapobiega to konfliktom prądowym między procesorem a pamięcią RAM.
- **Sygnał Sid:** Regularne impulsy 0001 (INC PC) w fazach Fetch potwierdzają, że sprzętowy licznik programu pracuje poprawnie również podczas operacji na pamięci.





Trzecia faza symulacji, udokumentowana na zrzutach Binary oraz Decimal, obejmuje końcowe instrukcje programu testowego (PC 12–14). W tej części zweryfikowano unikalną funkcjonalność procesora – modyfikację rejestrów bezpośrednio wewnątrz pliku rejestrów (in-place) przy wykorzystaniu jednostki sterującej Sid.

1. Operacje przesunięć i rotacji (REG_SHIFT – Opcode 101): Zgodnie z listą rozkazów, instrukcje z grupy REG_SHIFT wykorzystują 5-bitowe pole funkcji do sterowania modułem Sid.

- **PC = 12 (SHL R4):** Instrukcja logicznego przesunięcia rejestru R4 w lewo.
    - W widoku dziesiętnym zaobserwowano, że wartość w **R4** zmienia się z 17 na **34** (17×2).
    - Widok binarny potwierdza, że sygnał **Sid** przyjmuje w tym czasie wartość **0110** (kod operacji SHL).
- **PC = 13 (ROR R4):** Instrukcja rotacji bitowej rejestru R4 w prawo.
    - Wartość w rejestrze **R4** powraca z 34 do **17**.
    - Sygnał **Sid** zmienia się na **1001** (kod operacji ROR), co wymusza na pliku rejestrów przesunięcie cykliczne.

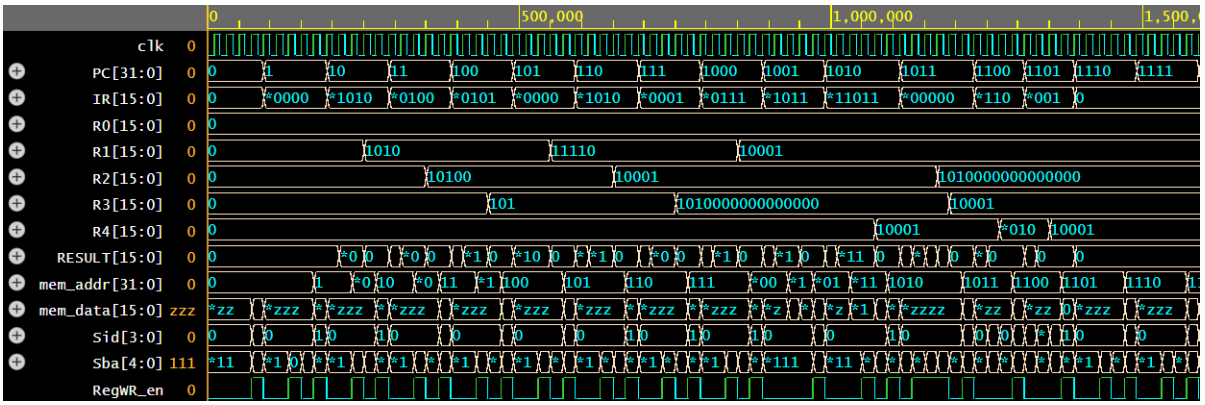
2. Kluczowa logika sterowania – RegWR_en i Sid:

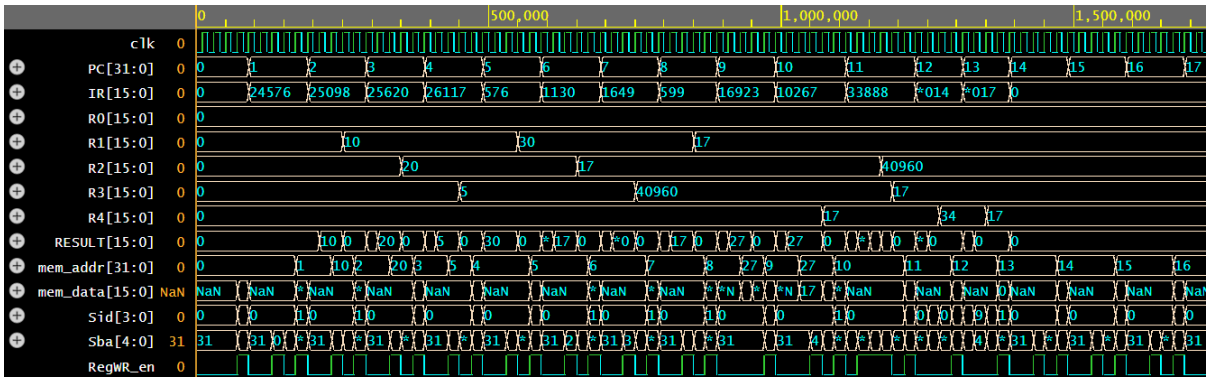
Na wykresie **Binary** zaobserwowano niezwykle istotny detal techniczny, o którym wspominano w założeniach projektowych:

- **Stan sygnału RegWR_en:** Podczas wykonywania operacji Sid (SHL/ROR), sygnał **RegWR_en (reg_write_enable) pozostaje w stanie niskim ('0')**.
- **Wniosek:** Potwierdza to, że modyfikacja danych zachodzi wewnątrz pliku rejestrów bez udziału głównej szyny zapisu procesora. Zapobiega to konfliktom na magistrali danych i pozwala na wykonanie operacji w jednym takcie zegara fazy Execute, co optymalizuje wydajność procesora wielocyklowego.

3. Zakończenie cyklu i weryfikacja stanu Done:

- **PC = 14:** Procesor pobiera kolejną instrukcję (IR = 0), co sygnalizuje koniec zdefiniowanego programu testowego.
- **Powrót do FETCH:** Automat stanów FSM powraca do fazy S_FETCH_1. Zgodnie z założeniami technicznymi, w tym momencie aktywowany jest sygnał **Done** (reprezentowany przez stan spoczynkowy automatu), co informuje o pomyślnym wykonaniu całej sekwencji rozkazów.





7.4. Logi przejść stanów

KERNEL: Time: 0 ps, Iteration: 1, Instance: /tb_processor_top/uut/UUT_ALU, Process: line__20.

EXECUTION:: NOTE : STATE: S_FETCH_2

EXECUTION:: Time: 70 ns, Iteration: 2, Instance: /tb_processor_top/uut/UUT_Control_Unit, Process: line__263.

EXECUTION:: NOTE : STATE: S_DECODE

EXECUTION:: Time: 90 ns, Iteration: 2, Instance: /tb_processor_top/uut/UUT_Control_Unit, Process: line__263.

EXECUTION:: NOTE : STATE: EXECUTE_MEM_ADDR

EXECUTION:: Time: 110 ns, Iteration: 2, Instance: /tb_processor_top/uut/UUT_Control_Unit, Process: line__263.

EXECUTION:: NOTE : STATE: S_WRITEBACK_ALU

EXECUTION:: Time: 130 ns, Iteration: 2, Instance: /tb_processor_top/uut/UUT_Control_Unit, Process: line__263.

EXECUTION:: NOTE : STATE: S_FETCH_1

EXECUTION:: Time: 150 ns, Iteration: 2, Instance:

```
/tb_processor_top/uut/UUT_Control_Unit,  Process: line__263.
# EXECUTION:: NOTE   : STATE: S_FETCH_2
# EXECUTION:: Time: 170 ns,  Iteration: 2,  Instance:
/tb_processor_top/uut/UUT_Control_Unit,  Process: line__263.
# EXECUTION:: NOTE   : STATE: S_DECODE
# EXECUTION:: Time: 190 ns,  Iteration: 2,  Instance:
/tb_processor_top/uut/UUT_Control_Unit,  Process: line__263.
# EXECUTION:: NOTE   : STATE: EXECUTE_MEM_ADDR
# EXECUTION:: Time: 210 ns,  Iteration: 2,  Instance:
/tb_processor_top/uut/UUT_Control_Unit,  Process: line__263.
# EXECUTION:: NOTE   : STATE: S_WRITEBACK_ALU
# EXECUTION:: Time: 230 ns,  Iteration: 2,  Instance:
/tb_processor_top/uut/UUT_Control_Unit,  Process: line__263.
# EXECUTION:: NOTE   : STATE: S_FETCH_1
# EXECUTION:: Time: 250 ns,  Iteration: 2,  Instance:
/tb_processor_top/uut/UUT_Control_Unit,  Process: line__263.
# EXECUTION:: NOTE   : STATE: S_FETCH_2
# EXECUTION:: Time: 270 ns,  Iteration: 2,  Instance:
/tb_processor_top/uut/UUT_Control_Unit,  Process: line__263.
# EXECUTION:: NOTE   : STATE: S_DECODE
# EXECUTION:: Time: 290 ns,  Iteration: 2,  Instance:
/tb_processor_top/uut/UUT_Control_Unit,  Process: line__263.
# EXECUTION:: NOTE   : STATE: EXECUTE_MEM_ADDR
# EXECUTION:: Time: 310 ns,  Iteration: 2,  Instance:
/tb_processor_top/uut/UUT_Control_Unit,  Process: line__263.
# EXECUTION:: NOTE   : STATE: S_WRITEBACK_ALU
# EXECUTION:: Time: 330 ns,  Iteration: 2,  Instance:
/tb_processor_top/uut/UUT_Control_Unit,  Process: line__263.
```

Oprócz analizy graficznej, zweryfikowano poprawność pracy jednostki sterującej poprzez analizę logów tekstowych generowanych podczas symulacji. Logi te pozwalają na śledzenie sekwencji stanów automatu FSM.

Kluczowe wnioski z analizy logów:

1. **Sekwencyjność:** Każda instrukcja rozpoczyna się od stanów S_FETCH_1 i S_FETCH_2, co potwierdza stały rytm pracy procesora.
2. **Poprawność dekodowania:** Po stanie S_DECODE automat bezbłędnie wybiera ścieżkę wykonawczą. Przykładowo, dla instrukcji arytmetycznych następuje przejście do S_EXECUTE_ALU, a dla instrukcji pamięciowych do S_EXECUTE_MEM_ADDR.
3. **Powrót do bazy:** Po każdym cyklu zapisu (S_WRITEBACK), automat każdorazowo powraca do stanu S_FETCH_1, co udowadnia, że procesor nie „zawiesza się" i jest gotowy do pobrania kolejnego rozkazu.

8. Podsumowanie i wnioski

8.1. Realizacja celów projektu

W ramach niniejszego projektu pomyślnie zaprojektowano i zaimplementowano w pełni funkcjonalny model 16-bitowego wielocyklowego procesora. Wszystkie założone cele techniczne oraz wymagania rozszerzone dla zespołu 4-osobowego zostały osiągnięte:

- Architektura wielocyklowa: Opracowano stabilny automat stanów (FSM), który zarządza cyklem rozkazowym w podziale na fazy (Fetch, Decode, Execute, Writeback). Gwarantuje to poprawne wykonywanie instrukcji bez występowania stanów nieustalonych, a powrót do stanu początkowego po zakończeniu operacji jest równoznaczny z powrotem do fazy S_FETCH_1, co oznacza gotowość do podjęcia kolejnych obliczeń.
- Wydajna jednostka ALU: Moduł obsługuje pełen zestaw 24 operacji arytmetyczno-logicznych. Implementacja zapewnia poprawną aktualizację kompletu flag stanu: C (przeniesienie), Z (zero), N (negacja) oraz V(Overflow).
- Rozbudowany plik rejestrów: Zaimplementowano 20 rejestrów, co stanowi rozszerzenie bazy o 8 jednostek (zgodnie z wytycznymi: +2 rejestry na każdego członka zespołu).
- Unikalny system Sid: Wprowadzono mechanizm operacji „in-place", który umożliwia inkrementację PC/SP oraz przesunięcia bitowe (SHL, SHR, ROL,

ROR) bezpośrednio w pliku rejestrów. System ten aktywuje się przy niskim stanie sygnału **RegWR_en**, co zapobiega konfliktom na magistrali i znacząco przyspiesza operacje sterujące.

8.2. Wnioski techniczne

Analiza przeprowadzonych symulacji oraz proces tworzenia kodu VHDL pozwoliły na wyciągnięcie następujących wniosków:

- **Modułowość architektury:** Podział procesora na niezależne bloki (ALU, CU, Rejestry) ułatwił proces debugowania i weryfikacji poszczególnych etapów pracy.
- **Efektywność operacji Sid:** Wykorzystanie dedykowanych ścieżek dla inkrementacji i zsuwów wewnątrz rejestrów pozwoliło na skrócenie czasu wykonania tych instrukcji (brak konieczności angażowania głównej szyny danych i ALU).
- **Zarządzanie pamięcią:** Zastosowanie rejestrów buforowych MAR i MBR okazało się kluczowe dla zapewnienia stabilności danych na szynie $inout$ podczas operacji odczytu i zapisu do pamięci zewnętrznej.

8.3. Konkluzja

Projekt stanowi solidną bazę do zrozumienia niskopoziomowej architektury komputerów oraz zasad projektowania systemów cyfrowych w językach opisu sprzętu. Uzyskane wyniki symulacji w pełni potwierdzają poprawność przyjętych założeń projektowych oraz stabilność zaimplementowanej jednostki centralnej.